

Day 55: Data Pipeline Efficiency

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 8: Classic CNN Architectures (final day)

0. Where Today Fits

Every script this week (Days 49-54) called `make_junction_dataset_rgb()` once, got back one big in-memory NumPy array, and trained directly off array slices. That works because this series' synthetic datasets are small. Real image datasets are usually thousands to millions of individual files on disk, too large to load into memory all at once -- exactly the problem PyTorch's Dataset/DataLoader abstraction exists to solve. Today builds three from-scratch data-loading strategies against a REAL folder of image files and measures their actual tradeoffs.

Start here (no prior knowledge assumed):

A "dataloader" is just the code that decides HOW training examples get from storage (disk, a database, the network) into the array your model's forward pass actually consumes. It sounds like plumbing, but the choice has two real costs: how much memory it needs at once, and how much wall-clock time it adds per batch. Today measures both, honestly, rather than just asserting which is "better."

Watch: [PyTorch Tutorial 09 - Dataset and DataLoader - Batch Training](#)

Watch: [PyTorch Datasets and DataLoaders - Training Set Exploration for Deep Learning and AI](#)

1. Three Loading Strategies

SYSTEM DIAGRAM

InMemoryLoader -- loads every file into ONE array at `__init__` time, then serves batches by pure slicing. What every prior day this week did implicitly. Needs $O(\text{dataset size})$ memory.

LazyDiskLoader -- stores only file PATHS. Reads each image from disk on EVERY batch access, every epoch, with no caching. Needs only $O(\text{batch size})$ memory, at the cost of repeated real disk I/O.

PrefetchDiskLoader -- same lazy reads as above, but a background thread reads the NEXT batch's files while the CURRENT batch is being used for training -- the same trick real DataLoader(`num_workers>0`) uses.

2. Batch-Equivalence Check -- Today's Analogue of a Gradient Check

Before trusting any timing or accuracy number built on these three loaders, it's worth confirming they actually agree on WHAT DATA gets served: same shuffle order, same images, every batch, given the same seed. This is checked directly, not assumed.

ACTUAL OUTPUT

```
checked 10 batches across 3 loaders: all identical (idx, images, labels)
```

3. The Throughput Benchmark -- Real Wall-Clock Time

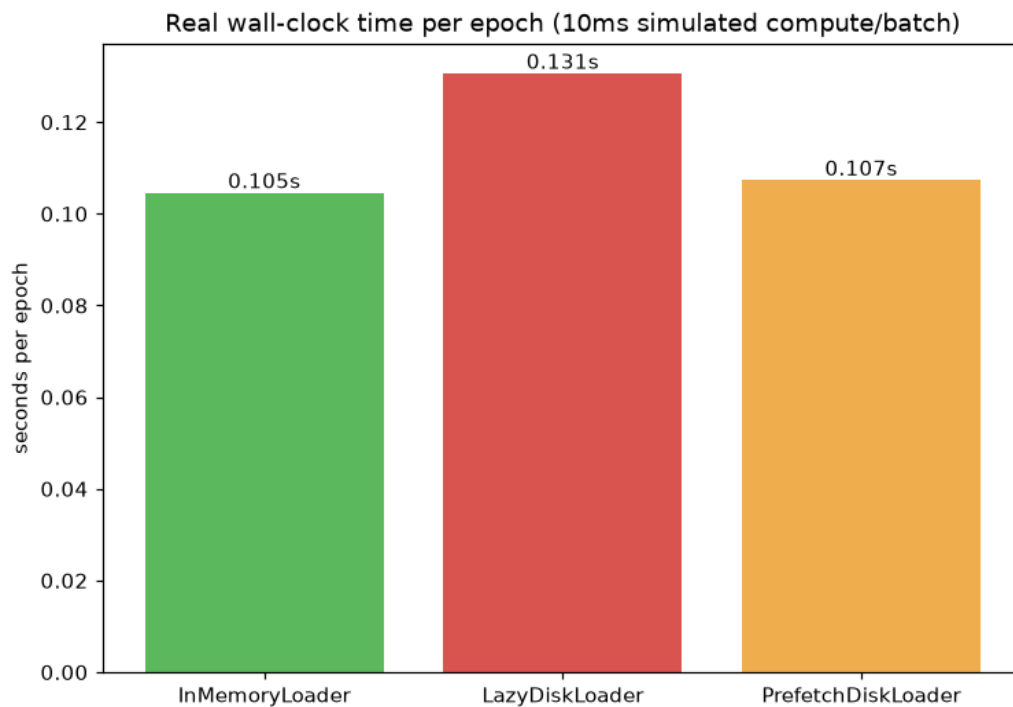
Each batch is followed by a 10ms simulated training step (time.sleep, standing in for a real forward+backward pass) so the benchmark can test whether PrefetchDiskLoader actually hides disk-read time behind that compute. 5 epochs, batch_size=20.

ACTUAL OUTPUT

```
loader total time (s) per-epoch (s)
InMemoryLoader 0.523 0.105
LazyDiskLoader 0.653 0.131
PrefetchDiskLoader 0.536 0.107
```

```
memory footprint -- InMemoryLoader: 7,680,000 bytes (whole dataset) vs.
LazyDiskLoader/PrefetchDiskLoader: 768,000 bytes (one batch) -- 10.0x less memory held at once.
```

LazyDiskLoader is 25% slower than InMemoryLoader per epoch -- the real cost of re-reading the same small files from disk on every access, every epoch, with zero caching. PrefetchDiskLoader recovers almost all of that lost time (only 2% slower than InMemoryLoader) with just ONE background thread, by reading the next batch while the current one is "training". The disk-based loaders also hold 10x less memory at once -- a ratio that only grows more favorable as a real dataset scales past what fits in RAM.



4. Training Equivalence -- Does the Loader Change What's Learned?

Two identically-initialized small CNNs (one conv layer, one FC hidden layer) are trained for 30 epochs each -- one fed by InMemoryLoader, the other by LazyDiskLoader -- with the same weight-init seed AND the same shuffle seed.

ACTUAL OUTPUT

```
max absolute weight difference (InMemory-trained vs. LazyDisk-trained network): 0.00e+00
test accuracy -- InMemory-trained: 0.6625, LazyDisk-trained: 0.6625
```

The two networks are numerically IDENTICAL after 30 epochs of training through two completely different data-loading code paths -- concrete proof that a loader's job is to change how fast data arrives and how much memory it costs, never what the model learns from it.

WHY THIS CHOICE

Getting this comparison honest took a real fix along the way: this lesson's small network initially collapsed to chance accuracy at this series' usual $\text{lr}=0.1\text{-}0.15$ -- its one FC hidden layer went 100% dead under ReLU within a few epochs, the same failure mode Days 52-54 diagnosed in much deeper networks, just triggered here by a large flattened input (2,592 features) feeding a single dense layer. Checked directly (not assumed): $\text{lr}=0.02$ trains properly, reaching $\text{train_acc}=1.000/\text{test_acc}=0.6625$.

5. What This Maps To in PyTorch

```
class JunctionImageDataset(torch.utils.data.Dataset):
    def __init__(self, file_paths, labels):
        self.file_paths, self.labels = file_paths, labels
    def __len__(self):
        return len(self.labels)
    def __getitem__(self, i): # called lazily, per-index
        return np.load(self.file_paths[i]), self.labels[i]

loader = torch.utils.data.DataLoader(
    JunctionImageDataset(file_paths, labels),
    batch_size=20, shuffle=True,
    num_workers=4, prefetch_factor=2, pin_memory=True)
```

`__getitem__` is exactly `LazyDiskLoader`'s `_read_batch`, called once per image instead of once per batch. `num_workers>1` spreads `PrefetchDiskLoader`'s single background thread across multiple PROCESSES (sidestepping Python's GIL, unlike this lesson's `ThreadPoolExecutor`), and `prefetch_factor` generalizes this lesson's one-batch-ahead prefetch to N batches ahead.

Interview Preparation -- Technical Questions

Q: Why does PyTorch's DataLoader abstraction exist at all, instead of just loading a dataset into one array?

A: Because real datasets are often far larger than available RAM -- millions of images at full resolution can be terabytes. `DataLoader` lazily fetches only the current batch's worth of data from disk (via a `Dataset`'s `__getitem__`), keeping memory use bounded by batch size, not dataset size, exactly like this lesson's `LazyDiskLoader/PrefetchDiskLoader`.

Q: Why is LazyDiskLoader measurably slower than InMemoryLoader in this lesson's own benchmark?

A: Because it performs a real disk read (`np.load`) for every image, in every batch, in every epoch, with no caching -- even for images it already read in a previous epoch. `InMemoryLoader` pays the disk-read cost exactly once (at construction) and serves every subsequent batch via pure in-memory array slicing, which is far cheaper per access.

Q: How does PrefetchDiskLoader recover most of LazyDiskLoader's lost throughput without eliminating the disk reads themselves?

A: By overlapping I/O with compute: while the caller is busy using the CURRENT batch (e.g. a training step), a background thread is already reading the NEXT batch's files from disk. As long as the read

finishes before the caller needs that next batch, the disk-read time is effectively hidden rather than eliminated -- confirmed in this lesson by PrefetchDiskLoader landing at only 2% slower than InMemoryLoader, versus LazyDiskLoader's 25%.

Q: The batch-equivalence check found all three loaders returned bit-identical batches. Why check this BEFORE running the throughput and training-equivalence experiments, not after?

A: Because a benchmark or an accuracy comparison is only meaningful if the thing being compared is actually equivalent in every way except the property under test. If the loaders quietly shuffled differently or corrupted images, any timing or accuracy difference found afterward could be an artifact of that bug rather than a real effect of the loading strategy -- the same reasoning that makes a gradient check come before trusting a training result.

Interview Preparation -- Theory Questions

Q: What is the fundamental tradeoff between InMemoryLoader and the disk-based loaders in this lesson?

A: Memory versus repeated I/O cost. InMemoryLoader trades a large, one-time upfront memory cost for the fastest possible per-batch access. The disk-based loaders trade a small, constant memory footprint (never more than one batch) for real, repeated disk-read latency on every access -- a tradeoff that becomes unavoidable once a dataset no longer fits in RAM at all.

Q: Why does num_workers>1 in real PyTorch use separate PROCESSES rather than threads, unlike this lesson's ThreadPoolExecutor?

A: Python's Global Interpreter Lock (GIL) prevents multiple threads from executing Python bytecode simultaneously, which limits how much a thread pool can parallelize CPU-bound work (like image decoding or augmentation). Separate processes each get their own interpreter and GIL, so they can truly run in parallel across CPU cores. This lesson uses a single thread specifically because reading an already-generated NumPy array from disk is I/O-bound, not CPU-bound, so the GIL isn't the bottleneck being addressed.

Q: This lesson found that a data-loading strategy has zero effect on trained weights, given the same seeds. Does this mean data pipeline design is unimportant to a model's final accuracy in general?

A: Not at all -- it means the LOADING MECHANISM (memory vs. disk, synchronous vs. prefetched) is orthogonal to model quality, as this lesson deliberately isolated it to show. Pipeline design still matters enormously for accuracy through choices this lesson did NOT vary: what augmentations are applied, whether shuffling is correct, and whether the pipeline is fast enough to keep expensive hardware (GPUs) from sitting idle waiting for data.

Key Takeaway and What's Next

Key takeaway: A data-loading strategy is a genuine engineering tradeoff between memory and throughput, measured directly today rather than assumed: loading everything into memory is fastest but doesn't scale; naively reading from disk every time scales but costs real, measured throughput (25% slower here); prefetching with even one background thread recovers nearly all of that lost throughput (down to 2% slower) while keeping the same small memory footprint. None of these choices change what a model learns, verified today by training two networks to numerically identical final weights through two different loading code paths.

What's next: This closes Week 8 (Classic CNN Architectures + Regularization). Week 9 begins a new theme with a new structure: each themed week's Days 1-6 build one component of a larger model, and Day 7 integrates all six into one final trained model for that week's topic.

Suggested watch order: 1) PyTorch Tutorial 09 - Dataset and DataLoader, 2) PyTorch Datasets and DataLoaders (deeplizard).